## 第五章 5.3 GPIO编程方法

- **系统时钟—Systick：（第二章相关）**
- ARM Cortex-M4处理器的系统定时器SysTick是一个24位的倒计时定时器，用于在每隔一定的时间产生一个中断，常用于实时操作系统RTOS节拍定时（即所谓的心跳），或者充当简单的计数器用来测量完成时间和使用时间。SysTick也可工作于系统睡眠模式下。SysTick属于**单调递减**计数器，每个时钟递减一次，从重载值连续递减到0。SysTick简单易用、配置灵活，还具有写入即清零、过零自动重载等灵活的控制机制。SysTick通过3个寄存器管理：
  - 控制和状态寄存器CTRL;：用于配置时钟数值、使能计数器功能、使能SysTick中断和确定计数器状态。
  - 重载值寄存器LOAD：计数器的重载值，用于每当计数器过零时自动重载。初始化SysTick时填写该值。
  - 当前值寄存器VAL：计数器的当前值。通过写入任意值来清零该寄存器。

33

## 第五章 5.3 GPIO编程方法

- **系统时钟—Systick：**
- 内核外设**Systick 寄存器的地址范围是**0xE000 E010 ～0xE000 E01F，位于System区(0xE000 0000～0xFFFF FFFF)，有4个寄存器

| | | |
|---|---|---|
| 0xE000E010 ~0xE000E01F | 系统定时器 SysTick | SYST_CSR：CTRL ：SysTick的控制和状态<br>SYST_RVR：LOAD： 存放SysTick重载值，24位<br>SYST_CVR：VAL： SysTick运行的当前值，24位<br>SYST_CALIB：CALIB： SysTick的校准值 |

- core_cm4.h

```
#define SCS_BASE        (0xE000E000UL)
#define SysTick_BASE    (SCS_BASE + 0x0010UL)
define SysTick          ((SysTick_Type   *) SysTick_BASE  )
```

```
typedef struct
{
  __IO uint32_t  CTRL;
  __IO uint32_t  LOAD;
  __IO uint32_t  VAL;
  __IO uint32_t  CALIB;
}SysTick_Type
```

34

## 第五章 5.3 GPIO编程方法

- **系统时钟—Systick** 为了更好应用**Systick功能，**以选择STM32F40_41xxx为例，说明如何得到其定时大小。
- 首先是SYSCLK和HCLK的获得
  - 从系统启动代码**startup_stm32f40_41xxx.s**中可看出，在进入 main 函数之前，系统调用 void SystemInit(void) 函数。
  - 在**system_stm32f4xx.c**文件中**void SystemInit(void)**又调用了**static void SetSysClock(void));**该函数也在该文件中，使能HSE，定义HSE_VALUE的值，对PLL初始化，等待稳定，由SYSCLK = (HSE_VALUE / PLL_M) * PLL_N / PLL_P得到系统时钟。
  - （一般用户使用外部高速时钟源HSE），如果外部时钟 HSE 启动失败（如无外部晶振元件），系统会使用内部时钟默认配置，即缺省 HCLK=SYSCLK/1=168MHz，PCLK2=HCLK/2=84MHz，PCLK1=HCLK/4=42MHz。
  - 下一步 调用**stm32f4xx_rcc.c**文件中 void RCC_GetClocksFreq(RCC_ClocksTypeDef* RCC_Clocks) 函数，如：在usermain.c例程中，

RCC_GetClocksFreq(&RCC_Clocks)去获得HCLK_Frequency大小。

其中结构体RCC_ClocksTypeDef包含了SYSCLK、HCLK、PCLK1、PCLK2

由此得到各项数值

```
结构体
typedef struct {
  uint32_t SYSCLK_Frequency;
  uint32_t HCLK_Frequency;
  uint32_t PCLK1_Frequency;
  uint32_t PCLK2_Frequency;
}RCC_ClocksTypeDef;
```

35

## 第五章 5.3 GPIO编程方法

- **系统时钟—Systick （展示STD方法中代码SYSCLK的获得 ）：**
- 缺省HSE_VALUE=25MHz，#define PLL_M 25，PLL_N=336，PLL_P=2， PLL_Q=7，正好168M
- 改为HSE_VALUE=8MHz，#define PLL_M 8，PLL_N=336，PLL_P=2， PLL_Q=7，可得168M
- PLL_VCO = (HSE_VALUE / PLL_M) * PLL_N 和 SYSCLK = PLL_VCO / PLL_P；
- 所以SYSCLK = (HSE_VALUE / PLL_M) * PLL_N / PLL_P

```
stm32f4xx.h中定义HSE_VALUE
#if defined(STM32F40_41xxx) || defined(STM32F427_437xx) . . .
 #if !defined  (HSE_VALUE)
  #define HSE_VALUE    ((uint32_t)25000000) /*!< Value of the External oscillator in Hz */
 #endif /* HSE_VALUE */
#elif defined (STM32F412xG) || defined(STM32F413_423xx) || defined(STM32F446xx)
 #if !defined  (HSE_VALUE)
  #define HSE_VALUE    ((uint32_t)8000000) /*!< Value of the External oscillator in Hz */
 #endif /* HSE_VALUE */
#endif
```

```
在system_stm32f4xx.c中定义了
#if defined(STM32F40_41xxx) || . . .
 #define PLL_M   25
 . . .
#if defined (STM32F40_41xxx)
#define PLL_N    336
#define PLL_P    2
#endif  /* STM32F40_41xxx */
```

因此如果选8MHz，则要在usermain.h中添加或改为

```
#define HSE_VALUE    ((uint32_t)8000000)   （在stm32f4xx.h）
#define PLL_M    8     （在system_stm32f4xx.c）
```

注：在HAL方法中，由STM32CubeMX配置好时钟树

36

## 第五章 5.3 GPIO编程方法

- **下一步，将HCLK的大小设置给Systick的计时单位**
- 当配置完成系统时钟后，由RCC_GetClocksFreq(&RCC_Clocks)去获得RCC_Clocks.HCLK_Frequency。
- 然后配置**Systick，采用uint32_t SysTick_Config(uint32_t ticks)**函数，在core_cm4.h文件中

```
__STATIC_INLINE uint32_t SysTick_Config(uint32_t ticks)
{
 if ((ticks - 1UL) > SysTick_LOAD_RELOAD_Msk) { return (1UL); }

 SysTick->LOAD  = (uint32_t)(ticks - 1UL);
 NVIC_SetPriority (SysTick_IRQn, (1UL << __NVIC_PRIO_BITS) - 1UL);
 SysTick->VAL   = 0UL;
 SysTick->CTRL  = SysTick_CTRL_CLKSOURCE_Msk |
                  SysTick_CTRL_TICKINT_Msk   |
                  SysTick_CTRL_ENABLE_Msk;
 return (0UL);
}
```

| 位域 | 名称 | 功能描述 |
|---|---|---|
| 16 | COUNTFLAG | 计数标志位 (只读)：当计数器从1减成到0时，该位会被硬件自动置1，<br>读取该寄存器后，该位会自动清零，也可以通过向 SYST_CVR 寄存器写入任何值来清零。 |
| 2 | CLKSOURCE | 时钟源选择位 (读/写)：<br>0：使用外部参考时钟（Arm保留，具体取决于芯片设计）。<br>1：使用处理器内部时钟（Arm，即 HCLK）。<br>注意：该位为了获得更精确的控制，会选择内部时钟（CLKSOURCE=1）。 |
| 1 | TICKINT | 中断使能位 (读/写)：<br>0：计数器减到0时不产生SysTick异常（中断）请求。<br>1：计数器减到0时产生SysTick异常（中断）请求（前提等于1）。<br>注意：如果使用SysTick做操作系统心跳，此位必须置1，如果仅用于获得式超时，则可以置0。 |
| 0 | ENABLE | 计数器使能位 (读/写)：<br>0：关闭SysTick计数器。<br>1：开启SysTick计数器。注意：开启后，计数器立即从 SYST_RVR 里的值开始递减。 |

**ticks就是每次计时时间，例如HCLK=168MHz，例如SysTick_Config(RCC_Clocks.HCLK_Frequency / 1000);**

**当ticks=168MHz/1000时，即LOAD=168×10⁶÷1000=168000，从该值开始减一计时，每次从168000到0正好为1ms**

37

## 第五章 5.3 GPIO编程方法

- **系统时钟—Systick (STD方法中)：**
- **（用于延时函数）**
- 用于延时函数的代码展示：
- Delay( ) 函数的实质是多次中断Systick的LOAD值
- 根据向量表，SysTick 的中断号是SysTick_IRQn，入口地址是0x0000003C，中断处理函数为 void SysTick_Handler(void)，在该函数中每滴答一下递减一次。见附录2
- **在 HAL 方法中，** 采用 void HAL_Delay(uint32_t Delay)进行延时

**代码展示：**

| | 语句 | 注释 |
|---|---|---|
| 定义 | static __IO uint32_t uwTimingDelay;<br>RCC_ClocksTypeDef RCC_Clocks; | //定义时钟计数变量<br>//时钟源结构体 |
| 函数 | void Delay(__IO uint32_t nTime)<br>{<br>uwTimingDelay = nTime;<br>while(uwTimingDelay != 0);<br>} | //SysTick延时函数<br>//放在用户文件中 |
| 函数 | voidTimingDelay_Decrement(void)<br>{<br>if (uwTimingDelay != 0x00)<br>{<br>uwTimingDelay--;<br>}<br>} | //时钟计数变量递减完成延时 |
| 函数 | void SysTick_Handler(void)<br>{<br>TimingDelay_Decrement();<br>} | //SysTick的中断处理函数，放在文件stm32f4xx_it.c中 |

38